

Unit 3: Memory System Organization

- Overview of memory hierarchy: Primary, Secondary, Cache, Virtual memory
 - RAM and ROM: Characteristics and types
 - Cache memory: Mapping techniques (Direct, Associative, Set-Associative)
 - Cache coherence and write policies
 - Memory interleaving and access optimization techniques
 - Virtual memory: Paging and Segmentation
 - Memory allocation: Dynamic, Static, and Fragmentation
 - RAID: Levels, benefits, and performance considerations
 - Memory management hardware: MMU and TLB
 - Introduction to modern memory technologies: DRAM, SRAM, Flash
-

Memory Hierarchy Overview

The **memory hierarchy** is a structured arrangement of different types of memory in a computer system, organized based on **speed**, **cost**, and **capacity**. The goal is to balance performance and cost by using faster, smaller memory close to the CPU and slower, larger memory further away.

Memory Hierarchy Structure

1. **Registers (Fastest, Smallest, Most Expensive)** ○ Located inside the CPU.
 - Stores temporary data for immediate use by the processor.
2. **Cache Memory (Fast, Small, Expensive)** ○ High-speed memory closer to the CPU.
 - Stores frequently accessed data and instructions to reduce latency.
3. **Primary Memory (RAM - Moderate Speed, Moderate Cost, Limited Size)** ○ Also called **main memory**.
 - Stores data and instructions currently in use by the CPU.
 - Volatile: Loses data when the system is powered off.
4. **Secondary Memory (Slow, Inexpensive, Large)** ○ Non-volatile storage like hard drives, SSDs, and optical disks. ○ Used for long-

term data storage. ○ Data is transferred to primary memory when required.

5. **Virtual Memory (Logical Extension of RAM)** ○ A technique that uses part of the secondary memory (disk) to extend the apparent size of RAM.
 - Allows the system to handle larger programs or multiple applications simultaneously.

1. Primary Memory (Main Memory)

- **Description:**
 - Volatile memory (e.g., RAM - Random Access Memory). ○ Acts as the CPU's working space, storing data and instructions that are actively being used.
- **Features:**
 - Fast compared to secondary memory but slower than cache. ○ Limited capacity (e.g., 8 GB, 16 GB, 32 GB in modern systems).
 - Directly accessible by the CPU.
- **Examples:**
 - **DRAM (Dynamic RAM):** Slower but cheaper and widely used.
 - **SRAM (Static RAM):** Faster but more expensive (used in cache).

2. Secondary Memory

- **Description:**
 - Non-volatile storage used for permanent data storage.
 - Cannot be accessed directly by the CPU (requires I/O operations).
 - **Features:**
 - Large capacity (e.g., terabytes in modern systems). ○ Much slower than primary memory. ○ Inexpensive compared to RAM.
 - **Examples:**
 - **Hard Disk Drives (HDDs):** Mechanical storage with spinning platters.
 - **Solid-State Drives (SSDs):** Faster storage using flash memory.
 - **Optical Disks:** CDs, DVDs, and Blu-rays. ○ **USB Drives:** Portable storage devices.
 - **Use:** ○ Stores the operating system, software, and user data.
-

3. Cache Memory

□ **Description:**

- A small, high-speed memory located closer to the CPU (or built into it).
 - Stores frequently accessed data to reduce access time to main memory.
 - **Features:**
 - Significantly faster than RAM but smaller in size.
 - Divided into levels (L1, L2, L3), with L1 being the fastest and smallest.
 - **Levels:**
 - **L1 Cache:** Located inside the CPU, very fast but small (e.g., 32 KB).
 - **L2 Cache:** Larger but slower than L1 (e.g., 256 KB - 1 MB).
 - **L3 Cache:** Shared among CPU cores, slower than L2 (e.g., 8 MB+).
 - **Use:** ○ Minimizes the time the CPU spends waiting for data.
-

4. Virtual Memory

□ **Description:**

- A memory management technique that creates an illusion of having more RAM by using secondary storage (disk) as additional memory.
- Enabled by the operating system.
- **Features:**
 - Slower than RAM since it relies on disk access.
 - Used when physical RAM is insufficient.
 - Non-volatile since it uses the disk.
- **Mechanism:** ○ Divides memory into **pages** (fixed-size blocks). ○ Unused pages in RAM are swapped to a **paging file** (on disk) to free up space.
 - Frequently accessed pages are swapped back into RAM.
- **Use:** ○ Allows running large applications or multitasking even with limited RAM.

Comparison of Different Memory Types

Type	Speed	Cost (per bit)	Capacity	Volatility	Access by CPU
Registers	Fastest	Most expensive	Smallest (few bytes)	Volatile	Direct
Cache Memory	Very fast	Expensive	Small (KB to MB)	Volatile	Direct

Primary Memory	<i>Fast</i>	<i>Moderate</i>	<i>Moderate (GBs)</i>	<i>Volatile</i>	<i>Direct</i>
Type	Speed	Cost (per bit)	Capacity	Volatility	Access by CPU
Secondary Memory	<i>Slow</i>	<i>Cheapest</i>	<i>Largest (TBs+)</i>	<i>Non-volatile</i>	<i>Indirect (via I/O)</i>
Virtual Memory	<i>Slowest (diskbased)</i>	<i>Very cheap (uses disk)</i>	<i>Limited by disk size</i>	<i>Non-volatile (disk)</i>	<i>Indirect (via OS)</i>

RAM (Random Access Memory)

Definition:

RAM is a type of **volatile** memory used by the CPU to store data and instructions that are actively being accessed or processed. It provides temporary storage and loses its contents when the system is powered off.

Characteristics of RAM:

1. **Volatile Memory:** ○ Data is lost when power is turned off.
2. **Random Access:** ○ Any memory cell can be accessed directly, irrespective of its physical location.
3. **High Speed:**
 - Faster than secondary memory but slower than cache and registers.
4. **Temporary Storage:** ○ Stores data and instructions needed for currently running programs.
5. **Read-Write Memory:**
 - Allows both read and write operations.

Types of RAM:

1. **Dynamic RAM (DRAM):**
 - **Description:** Stores data in capacitors and requires periodic refreshing to maintain the data.
 - **Characteristics:**
 - Slower than SRAM.
 - Cheaper and higher capacity.
 - **Use:** Used as main memory in most computers.
 - **Variants:**

- *DDR (Double Data Rate) DRAM: Includes DDR2, DDR3, DDR4, DDR5 (modern systems).*
- *SDRAM (Synchronous DRAM): Operates in sync with the CPU clock.*
- 2. **Static RAM (SRAM):**
 - **Description:** Stores data using flip-flops, which do not require refreshing.
 - **Characteristics:**
 - *Faster than DRAM.*
 - *More expensive and less dense.*
 - **Use:** Used in CPU cache (L1, L2, L3).

Key Differences Between DRAM and SRAM:

<i>Feature</i>	<i>DRAM</i>	<i>SRAM</i>
<i>Speed</i>	<i>Slower</i>	<i>Faster</i>
<i>Cost</i>	<i>Cheaper</i>	<i>More expensive</i>
<i>Refresh</i>	<i>Requires periodic refreshing</i>	<i>No refresh required</i>
<i>Density</i>	<i>Higher (more memory per chip)</i>	<i>Lower</i>
<i>Use</i>	<i>Main memory</i>	<i>Cache memory</i>

ROM (Read-Only Memory)

Definition:

ROM is a type of **non-volatile** memory that stores data permanently. It is used for storing critical data and instructions that do not change, such as the firmware of a computer.

Characteristics of ROM:

1. **Non-Volatile Memory:**
 - *Retains its contents even when power is turned off.*
2. **Read-Only:**
 - *Data is written during manufacturing and cannot be modified (except for specific types).*
3. **Permanent Storage:** ○ *Stores essential data, like the system boot process (BIOS or UEFI).*
4. **Slower Than RAM:** ○ *Access speed is slower compared to RAM.*

5. **Used for Firmware:**

- Stores critical programs and data that do not require frequent updates.

Types of ROM:

1. **Masked ROM (MROM):** ○ **Description:** Programmed during manufacturing and cannot be modified. ○ **Use:** Used in devices where data never changes (e.g., embedded systems).
2. **Programmable ROM (PROM):** ○ **Description:** Can be programmed once using special equipment.
 - **Use:** Used in applications requiring custom data storage.
3. **Erasable Programmable ROM (EPROM):** ○ **Description:** Can be erased using ultraviolet light and reprogrammed.
 - **Use:** Used in early embedded systems where updates were rare.
4. **Electrically Erasable Programmable ROM (EEPROM):** ○ **Description:** Can be erased and reprogrammed using electrical signals. ○ **Use:** Common in modern systems for firmware updates (e.g., BIOS/UEFI).
5. **Flash Memory:** ○ **Description:** A modern, fast form of EEPROM used for non-volatile storage. ○ **Use:** Found in USB drives, SSDs, and memory cards.

Comparison between RAM and ROM

Aspect	RAM	ROM
Volatility	Volatile (data lost when power is off)	Non-volatile (data retained when power is off)
Access Type	Read and write	Typically read-only (modifiable for certain ROM types)
Speed	Faster	Slower
Purpose	Temporary storage for active data	Permanent storage for critical data
Examples	DRAM, SRAM	BIOS, Firmware, Flash Memory
Use Case	Running programs and processes	Boot instructions, device firmware

Aspect	DRAM (Dynamic RAM)	SRAM (Static RAM)	Flash Memory
--------	--------------------	-------------------	--------------

Speed	Moderate speed	Fastest of the three	Slowest (but faster than HDDs)
Aspect	DRAM (Dynamic RAM)	SRAM (Static RAM)	Flash Memory
- Access Time	~50-70 ns	~1-10 ns	~50-100 μ s (write) / ~25-50 μ s (read)
- Latency	Higher than SRAM	Lowest latency	Higher latency compared to DRAM
Cost	Lower cost (cheaper to manufacture)	Higher cost per bit	Low cost for high capacity
- Cost per GB	Moderate	Expensive (due to complexity)	Cheapest for nonvolatile memory
Power Consumption	Moderate	High (due to constant power to retain data)	Low (as it's nonvolatile)
- Standby Power	Requires refreshing, increasing power consumption	Requires continuous power to maintain data	Consumes power only during operations
- Active Power	Moderate to high (refresh cycles)	High during use, but stable	Low active power usage
Volatility	Volatile (loses data without power)	Volatile	Non-volatile (retains data without power)
Density	High density (more data per chip)	Low density (larger cells)	Very high density (ideal for storage)
Use Case	Main memory in computers and GPUs	CPU cache (L1, L2, L3), registers	Long-term storage (e.g., SSDs, USB drives)

1. Speed:

SRAM	VERSUS	DRAM
SRAM A type of semiconductor memory that uses bi-stable latching circuitry (flip flop) to store each bit Stands for Static Random Access Memory Very fast Does not require refresh cycles to retain data Requires refreshing, it has more complex circuitry and timing requirements Used for CPU cache Require minimum time to access data Complex structure - has flip flops Has a lower density Expensive		DRAM A type of random access semiconductor memory that stores each bit of data in a separate tiny capacitor within an integrated circuit Stands for Dynamic Random Access Memory Not as fast as SRAM Requires periodical refresh cycles to retain data Not as complex as SRAM Used for the computer's main memory Requires more time to access data Simple structure - has a transistor and a capacitor Has a higher density Less expensive Visit www.PEDIAA.com

- **SRAM** is the fastest because it uses flip-flops to store data, allowing instant access without requiring refreshing.
- **DRAM** is slower because it stores data in capacitors, which need periodic refreshing to maintain data integrity.
- **Flash Memory** is the slowest as it uses NAND-based architecture and involves writing/erasing data in blocks, making it unsuitable for frequent updates.

2. Cost:

- **SRAM** is the most expensive because it requires more transistors (6 per bit) and complex manufacturing.
- **DRAM** is cheaper due to its simpler design (1 transistor + 1 capacitor per bit).
- **Flash Memory** has the lowest cost per GB, as it is optimized for high-capacity storage.

3. Power Consumption:

- **SRAM** consumes more power because it needs continuous power to retain data and operate flipflops.
- **DRAM** consumes moderate power because it needs refreshing but less than SRAM.
- **Flash Memory** is the most power-efficient because it is non-volatile and only consumes power during read/write operations.

4. Use Cases:

- **SRAM:** Used in CPU cache and registers, where speed is critical.
- **DRAM:** Used as the main memory (RAM) in computers and GPUs, providing a balance of speed, cost, and power.
- **Flash Memory:** Used in SSDs, USB drives, and memory cards for non-volatile storage.

Memory Interleaving and Access Optimization Techniques

Memory interleaving is a technique used to increase the access speed of memory by dividing memory into multiple banks that can be accessed simultaneously. This improves CPU performance by reducing waiting time for memory access.

1. Memory Interleaving

□ **Concept:**

- The memory is divided into several blocks or banks, and consecutive memory addresses are assigned to different banks.
- While one bank is busy handling one request, other banks can process different requests, thus enabling parallelism.

□ **Types of Interleaving:**

1. **Low-order Interleaving:**

- Consecutive memory addresses are mapped to consecutive memory banks. ▪
Example: Address 0 → Bank 0, Address 1 → Bank 1, Address 2 → Bank 2, and so on.

2. **High-order Interleaving:**

- Blocks of addresses are assigned to different banks.
- Example: Addresses 0–3 → Bank 0, Addresses 4–7 → Bank 1, and so on.

- **Advantages:** ○ Reduces memory latency by enabling parallel access.
 - Increases throughput by minimizing idle time for memory banks.

- **Applications:**
 - Used in high-performance computing and systems requiring fast memory access (e.g., GPUs).

2. Access Optimization Techniques

- **Cache Memory:** Frequently accessed data is stored in the cache to reduce latency.
- **Prefetching:** Anticipates future memory requests and loads data into faster memory (cache) before the CPU needs it.
- **Burst Mode:** Transfers multiple data words in a single operation to optimize memory bandwidth.
- **Bank Switching:** Switches between memory banks to allow efficient use of different memory modules.

Virtual Memory: Paging and Segmentation

Virtual memory is a memory management technique that provides the illusion of a large address space by using a combination of physical memory (RAM) and secondary storage (disk). 1. Paging

- **Concept:**
 - Divides the virtual address space into fixed-size blocks called **pages** and physical memory into fixed-size blocks called **frames**.
 - Pages are mapped to frames using a **page table**.
- **How It Works:**
 1. A process generates a virtual address.
 2. The virtual address is divided into a **page number** (used to locate the page table entry) and an **offset** (used to find the specific location within the page).
 3. The page table maps the page to a physical frame in memory.
 4. If the page is not in physical memory, a **page fault** occurs, and the operating system loads the required page from disk.
- **Advantages:**
 - Simplifies memory allocation by using fixed-size pages.
 - Reduces fragmentation (no external fragmentation).
- **Disadvantages:**
 - Introduces overhead due to page table management.
 - Suffers from internal fragmentation if pages are not fully utilized.

2. Segmentation

- **Concept:**
 - Divides the virtual address space into **variable-sized segments**, each representing a logical division of a program (e.g., code, data, stack).
 - Each segment has a **base address** and a **limit** that defines its size.
- **How It Works:**
 1. A process generates a virtual address consisting of a **segment number** and an **offset**.
 2. The segment number is used to locate the segment descriptor (base and limit).

3. The offset is added to the base to calculate the physical address.
 4. If the offset exceeds the limit, a segmentation fault occurs.
- **Advantages:**
 -
 - Supports logical organization of memory (e.g., separating code and data).
 - Allows sharing and protection of segments between processes.
 - **Disadvantages:**
 - Can suffer from external fragmentation.
 - More complex memory management compared to paging.

Comparison Between Paging and Segmentation

Aspect	Paging	Segmentation
Division	Fixed-size pages and frames	Variable-size segments
Address	Page number + Offset	Segment number + Offset
Fragmentation	Internal fragmentation	External fragmentation
Use Case	General-purpose memory management	Logical program structure

Memory Allocation: Static, Dynamic, and Fragmentation

Memory allocation refers to how memory is assigned to programs during execution. 1.

Static Memory Allocation

- **Definition:**
 - Memory is allocated during **compile time** and cannot be resized or reallocated at runtime.
- **Advantages:**
 - Simple and fast since memory size is predetermined.
 - No runtime overhead for allocation.
- **Disadvantages:**
 - Inefficient memory usage if the allocated memory is not fully utilized.
 - Cannot handle dynamic data structures.
- **Examples:**
 - Global variables, local variables with fixed sizes.

2. Dynamic Memory Allocation

- **Definition:**
 - Memory is allocated during **runtime** based on program needs.
- **How It Works:**
 - Functions like malloc, calloc, realloc, and free in C allocate and free memory from the heap.
- **Advantages:**
 - Flexible: Memory can grow or shrink as needed.

- Efficient memory usage compared to static allocation.
- **Disadvantages:**
 - Slower due to runtime overhead.
 - May cause memory leaks if allocated memory is not freed properly.
- **Examples:** ○ Data structures like linked lists, trees, and dynamically sized arrays.

3. Fragmentation

- Fragmentation occurs when memory is not used efficiently, resulting in wasted space. 1.

Internal Fragmentation:

- Occurs when allocated memory is larger than requested, leaving unused space within an allocated block.
- Example: Allocating a 4 KB page to a process that only needs 3.5 KB.

2. External Fragmentation:

- Occurs when free memory is divided into non-contiguous blocks, making it difficult to allocate large memory blocks.
- Example: Several small free blocks exist, but none is large enough to meet the memory request.

3. Solution to Fragmentation:

- **Compaction:** Rearranges memory to combine free blocks (reduces external fragmentation).
 - **Paging:** Eliminates external fragmentation by using fixed-size pages.
 - **Best-fit/Worst-fit Allocation:** Reduces internal fragmentation by better matching memory requests with available blocks.
-

Summary Table

Aspect	Static Allocation	Dynamic Allocation	Fragmentation
When Allocated	Compile time	Runtime	-
Flexibility	Fixed-size allocation	Adjustable at runtime	Causes inefficient memory use
Aspect	Static Allocation	Dynamic Allocation	Fragmentation
Efficiency	Fast and simple	More flexible but slower	Can be mitigated with paging or compaction

Examples	Global variables, fixed arrays	Linked lists, dynamic arrays	Internal and external fragmentation
-----------------	--------------------------------	------------------------------	-------------------------------------

1. Paging

Definition:

Paging is a memory management technique that divides the logical address space into fixed-size blocks called **pages** and the physical memory into fixed-size blocks called **frames**. Pages are mapped to frames in physical memory using a **page table**.

How Paging Works:

1. The virtual address is divided into two parts:
 - **Page Number (PN):** Identifies the page in the virtual address space.
 - **Page Offset (PO):** Specifies the location within the page.
2. The **page table** is used to map the page number to a physical frame number in memory.
3. When a program accesses a memory location:
 - The CPU extracts the page number and looks up the page table to find the corresponding frame.
 - The page offset is added to the base address of the frame to calculate the physical address.
4. If a page is not in memory (due to limited RAM), a **page fault** occurs, and the operating system loads the page from disk into memory.

Advantages of Paging:

- **No External Fragmentation:**
 - Since memory is divided into fixed-size blocks, there is no fragmentation outside the allocated pages.
- **Efficient Use of Physical Memory:**
 - Pages can be loaded into any available frame, allowing non-contiguous allocation.
- **Simplifies Memory Allocation:**
 - Allocates memory in fixed-sized frames, reducing complexity.

Challenges:

- **Internal Fragmentation:**
 - Occurs if a page is not fully utilized.
- **Page Table Overhead:**
 - Large processes require large page tables, increasing memory overhead.
- **Translation Overhead:**

- Every memory access involves translation from virtual to physical address, increasing latency (mitigated by a **TLB** - Translation Lookaside Buffer).
-

2. Segmentation

Definition:

Segmentation is a memory management technique that divides the logical address space into variable-sized **segments**, each representing a logical part of a program (e.g., code, data, stack). Each segment has a **base address** and a **limit**.

How Segmentation Works:

1. The virtual address is divided into two parts:
 - **Segment Number (SN)**: Identifies the segment in the logical address space.
 - **Segment Offset (SO)**: Specifies the location within the segment.
 2. The **segment table** is used to map the segment number to a base address in physical memory.
 3. When a program accesses a memory location:
 - The CPU extracts the segment number and offset.
 - It looks up the segment table to find the base address of the segment.
 - The offset is added to the base address to calculate the physical address.
 4. If the offset exceeds the segment limit, a **segmentation fault** occurs, ensuring memory safety.
-

Advantages of Segmentation:

- **Logical View of Memory:**
 - Segments correspond to logical parts of the program, making it easier to organize and protect memory.
- **Supports Sharing and Protection:**
 - Segments can be shared between processes (e.g., shared libraries) and protected (e.g., read-only code segments).
- **Dynamic Allocation:**
 - Allows segments to grow dynamically based on program needs.

Challenges:

- **External Fragmentation:**
 - Since segments are of variable size, free memory may become fragmented.
- **Complexity:**
 - Managing variable-sized segments and segment tables is more complex than paging.

Comparison between Paging and Segmentation

<i>Aspect</i>	<i>Paging</i>	<i>Segmentation</i>
<i>Division</i>	<i>Fixed-size pages and frames</i>	<i>Variable-sized segments</i>
<i>Address Components</i>	<i>Page Number + Page Offset</i>	<i>Segment Number + Segment Offset</i>
<i>Fragmentation</i>	<i>Internal fragmentation (unused page space)</i>	<i>External fragmentation (non-contiguous free space)</i>
<i>Logical View</i>	<i>Physical memory-oriented</i>	<i>Program-oriented (logical division)</i>
<i>Ease of Allocation</i>	<i>Easier due to fixed sizes</i>	<i>More complex due to variable sizes</i>
<i>Protection/Sharing</i>	<i>Limited (page-level protection)</i>	<i>Better (segment-level protection/sharing)</i>

How They Improve Memory Allocation

Paging:

- ***Efficient Use of Physical Memory:***
 - *By dividing memory into fixed-size blocks, paging ensures that free memory is utilized effectively, without external fragmentation.*
- ***Non-Contiguous Allocation:***
 - *Pages can be placed in any free frame, allowing processes to grow dynamically without requiring contiguous physical memory.*
- ***Swapping and Demand Paging:***
 - *Enables loading only the required pages into memory, reducing the need for excessive RAM.*

Segmentation:

- ***Logical Organization:***
 - *Dividing memory into logical segments like code, data, and stack ensures a structured memory allocation.*
- ***Dynamic Allocation:***
 - *Segments can grow or shrink based on program needs, making memory usage flexible.*
- ***Protection:***
 - *Segments can be protected individually, preventing one segment from overwriting another, enhancing security.*

Combining Paging and Segmentation

*Many systems (e.g., x86 architecture) use a **hybrid approach**:*

- ***Segmentation with Paging:***
 - *Logical address space is divided into segments, and each segment is further divided into pages.*
 - *This combines the flexibility of segmentation with the simplicity of paging, addressing the limitations of each approach.*